

20

Loop Design Patterns

The difference between an agent that runs once and a system that improves itself.



Rakesh Gohel

Everyone tunes the prompt. Almost nobody designs the loop.

Old way

Prompt → Response → Done

New way

Act → Observe → Evaluate → Adjust

Every one of the 20 patterns ahead is a variation on that single cycle.



01

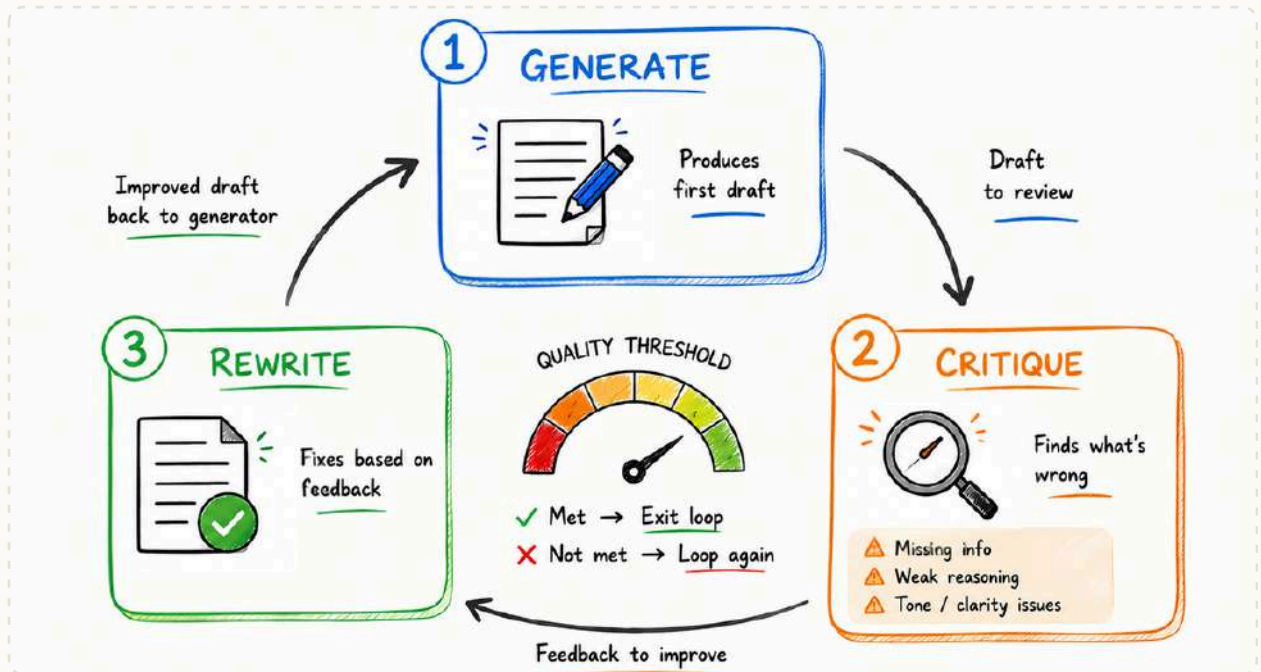
Quality loops

Make the output better before it leaves

1 Generate → Critique → Rewrite

A critic finds what the generator missed.

Generator drafts, a separate critic reviews, generator rewrites until quality holds. The model is never the best judge of its own output.



01

Quality loops

Make the output better before it leaves

2

Score-and-Retry

Retry until the score clears the bar.

Generate, score, retry if below threshold. Works best when quality is measurable — accuracy, format, factual correctness.

3

Multi-Critic

Four critics in parallel, all must pass.

Correctness, style, safety, and domain critics each review independently. Output only ships when every one is satisfied.

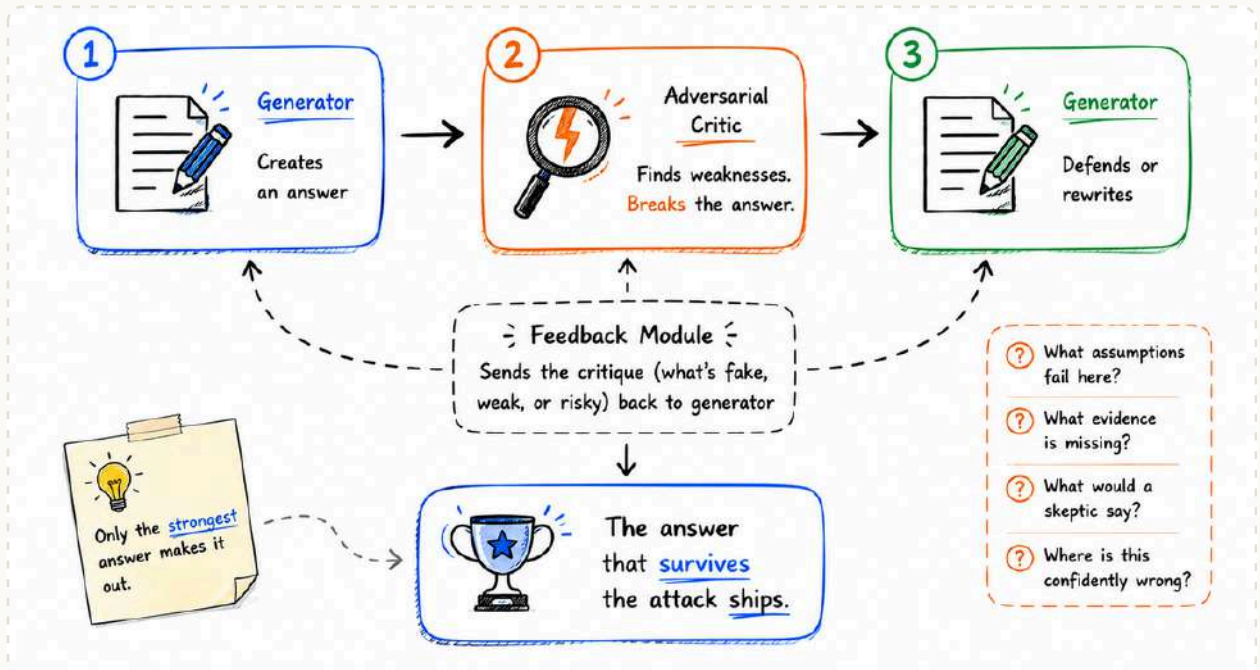


4

Adversarial Critique

The critic's only job is to break the answer.

Not improve it — break it. What assumptions fail? What would a skeptic say? The answer that survives the attack ships.

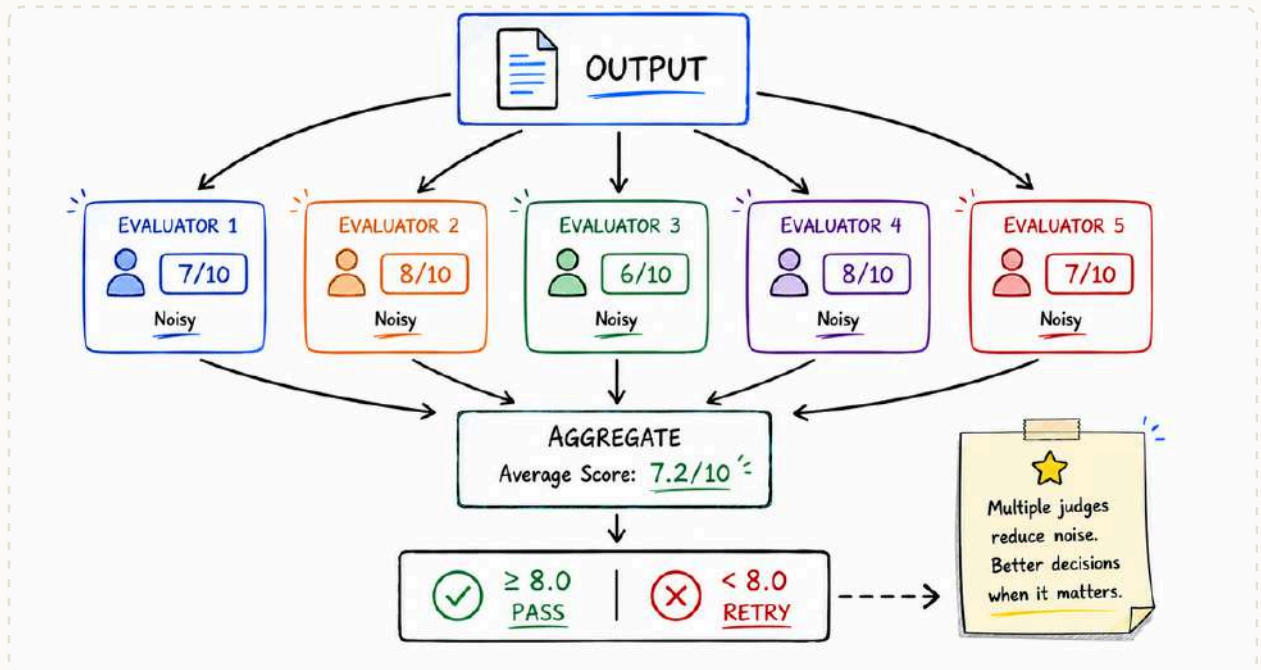


5

Judge Ensemble

Only high-consensus outputs advance.

One judge is noisy. Run the output through several evaluators and aggregate — critical when stakes are high.

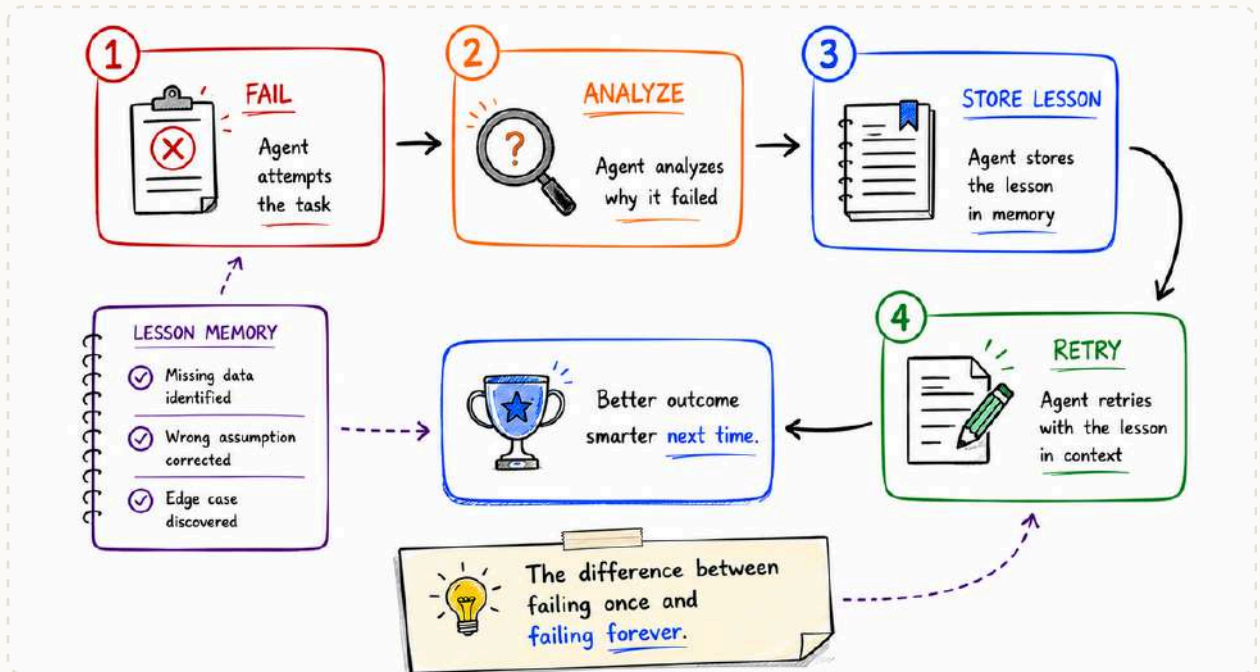


6

Reflexion

Fail, understand why, retry smarter.

Agent fails, analyzes the cause, stores the lesson, retries with it in context. The difference between failing once and failing forever.



7

Memory Update

The system reads its own history.

After every task, store what was decided, what happened, and what to change. Month six has read six months of its own runs.

8

Error Library

Never make the same mistake twice.

Store every failure. Search the library before a new task and apply the known fix first. The most underused pattern in production AI.



9

Success Pattern

Learn from wins, not just mistakes.

Most teams only log failures. Save the approach, context, and what worked — then retrieve it for similar tasks.

10

Memory Compression

Keep memory usable as it grows.

Unlimited memory is unusable memory. Compress many specific failures into one abstract pattern. Context stays fast.



03

Planning loops

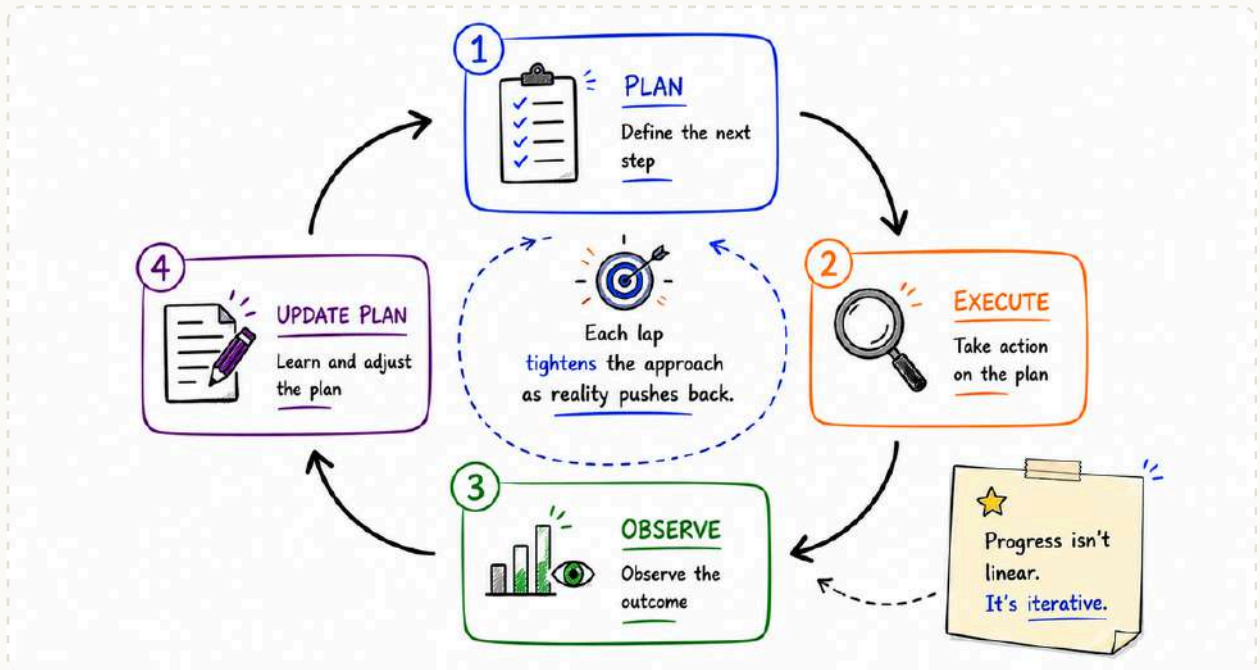
Adapt the plan when reality changes

11

Plan → Execute → Replan

Not a waterfall — a spiral.

Execute a step, observe the outcome, update the plan, continue. Each lap tightens the approach as reality pushes back.



03

Planning loops

Adapt the plan when reality changes

12 Dynamic Workflow

The pipeline picks its own shape.

Fixed pipelines break on edge cases. Dynamic ones branch on results at runtime — routing, adaptive research, support triage.

13 Goal Decomposition

Break it down until it's executable.

Large goal into subgoals into tasks into steps. Keep decomposing until each unit is small enough to run in a single call.



03

Planning loops

Adapt the plan when reality changes

14

Progress Evaluation

Check whether you're actually getting closer.

Every N steps, stop and ask: closer to the goal? If not, change strategy, tools, or plan — don't execute blindly.

15

Constraint Satisfaction

Not done until every rule passes.

Keep improving until budget, quality, latency, tone, and accuracy all hold at once. Very common in production systems.

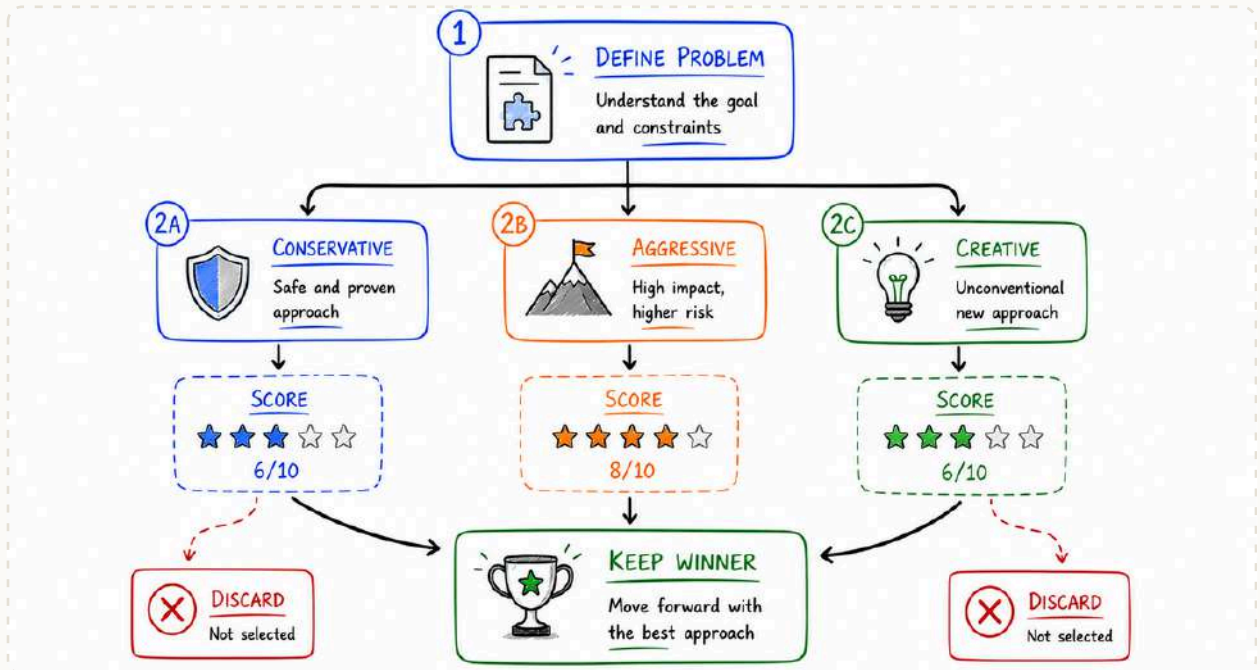


16

Branch-and-Explore

Try several paths, keep the best.

Generate conservative, aggressive, and creative approaches in parallel. Score them, keep the winner, discard the rest.

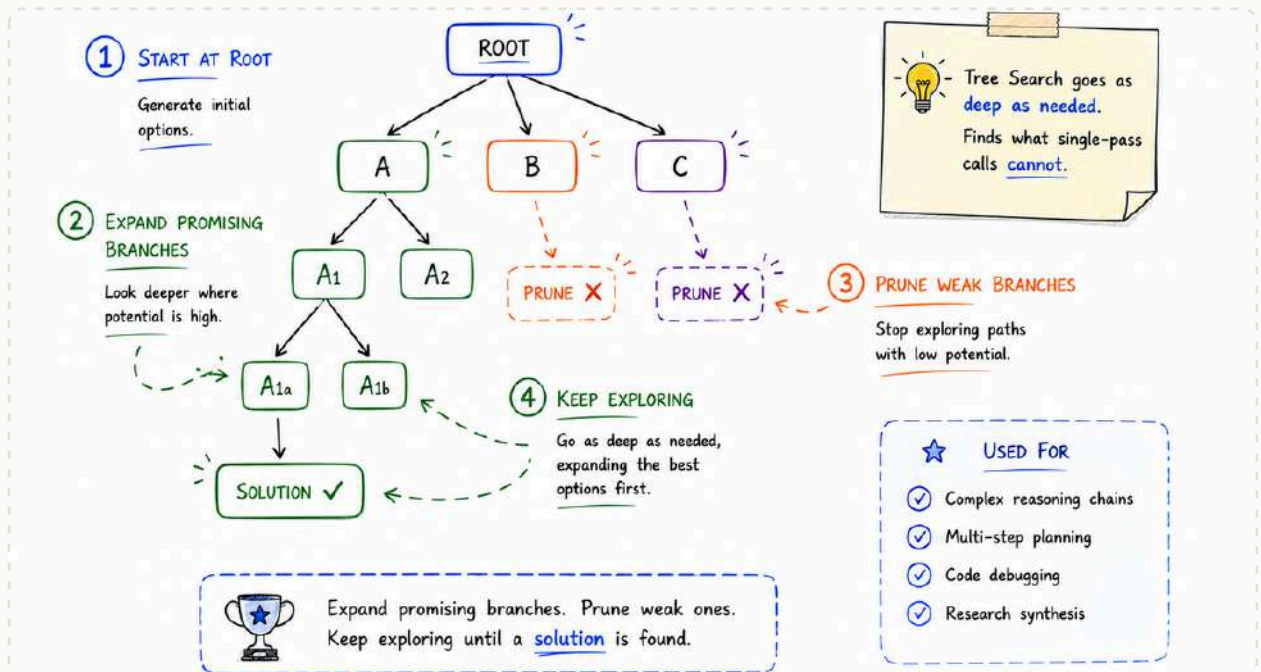


17

Tree Search

Go as deep as the problem needs.

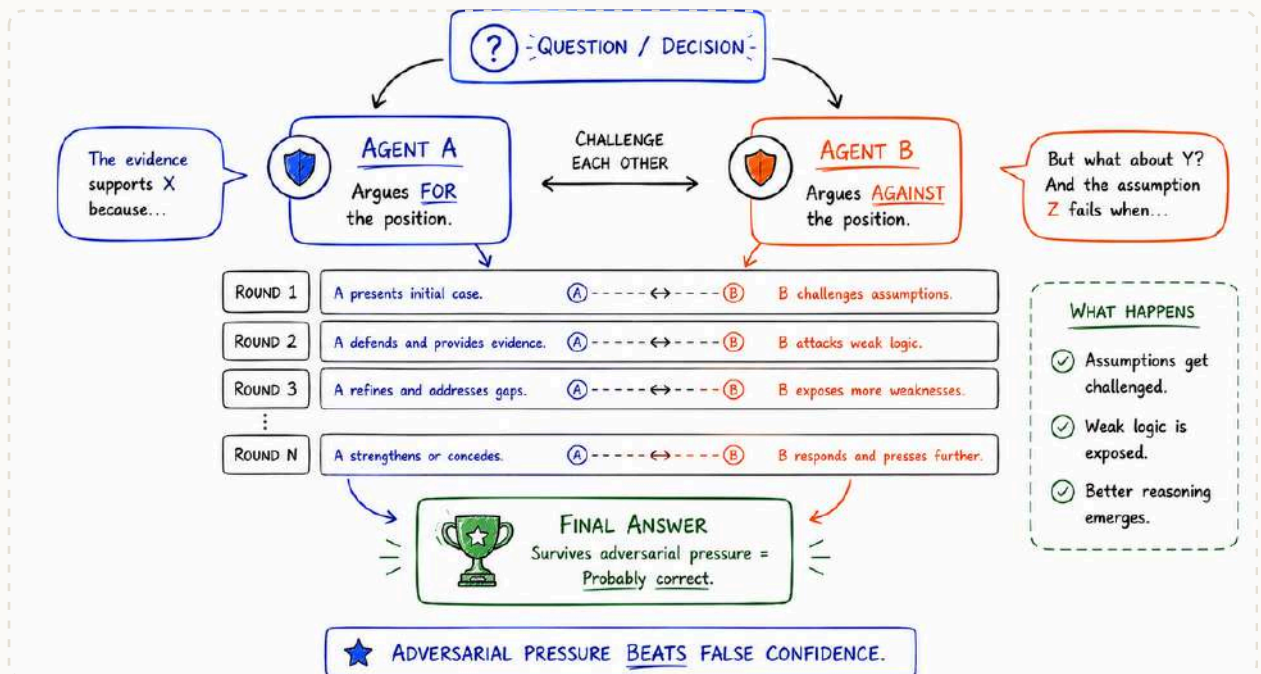
Expand promising branches, prune weak ones, keep exploring until a solution is found. Finds what single-pass calls cannot.



18 Debate

The answer emerges through disagreement.

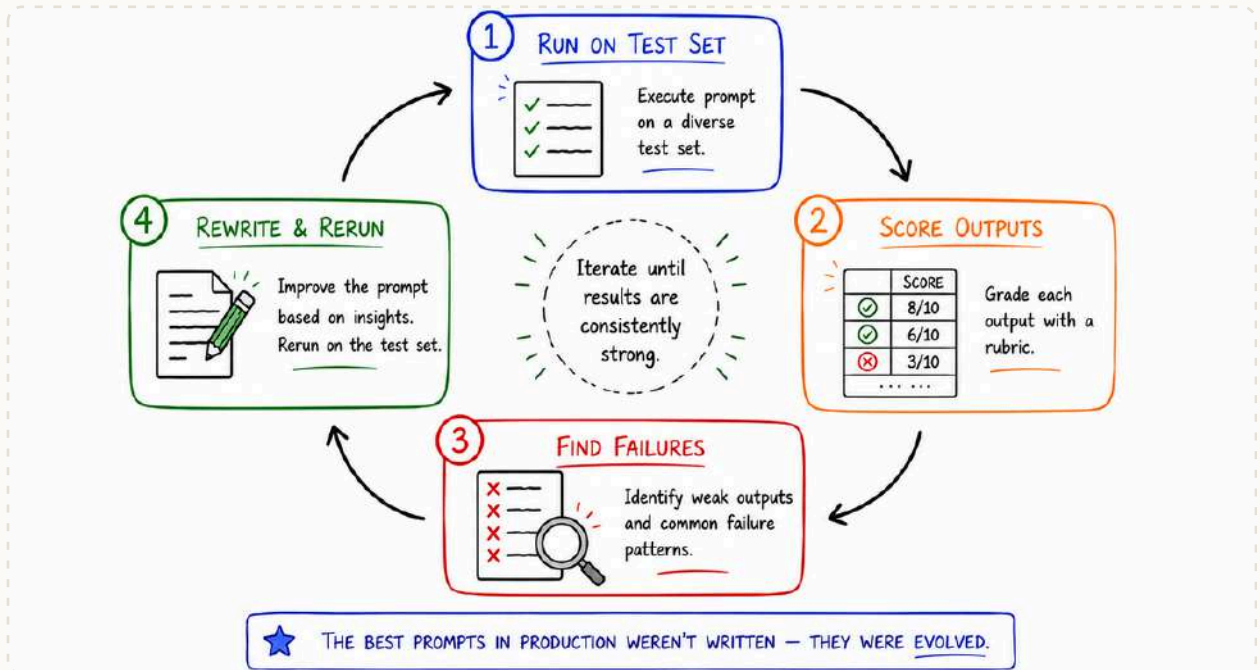
Two agents argue opposite positions. Each round challenges assumptions and exposes weak logic. Adversarial pressure beats false confidence.



19 Prompt Optimization

The prompt rewrites itself.

Run it on a test set, score outputs, find failures, rewrite, rerun. The best prompts in production weren't written — they were evolved.



05

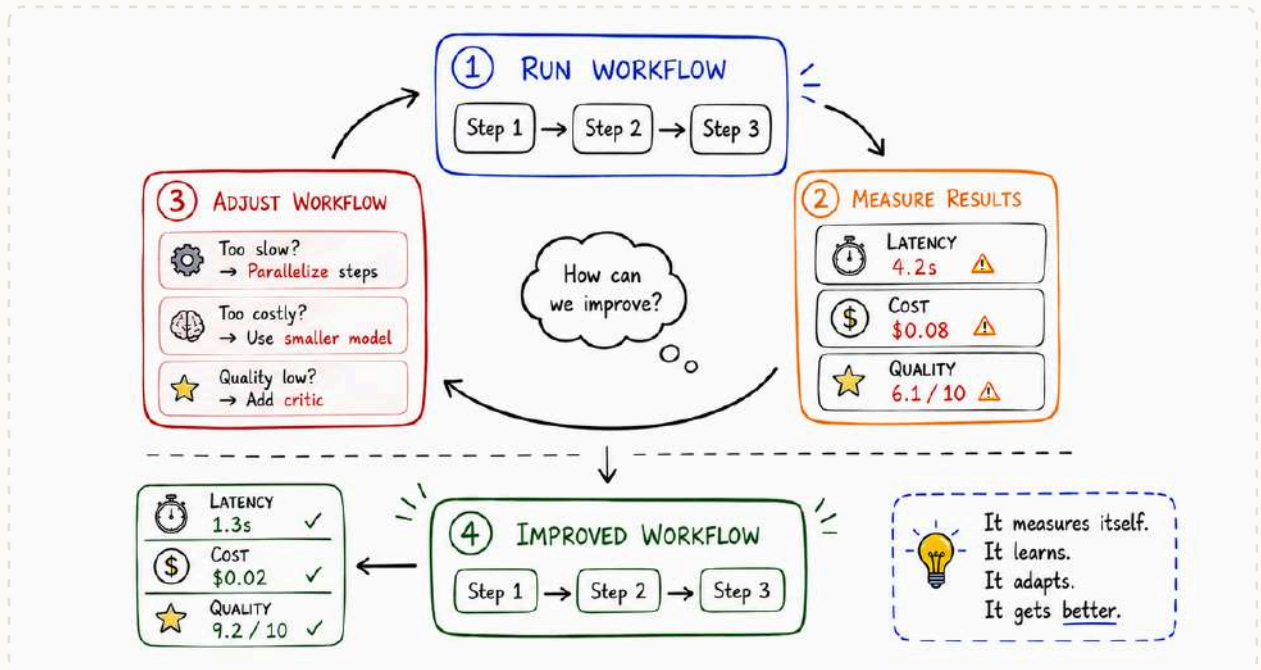
System optimization loops

The loop improves the loop

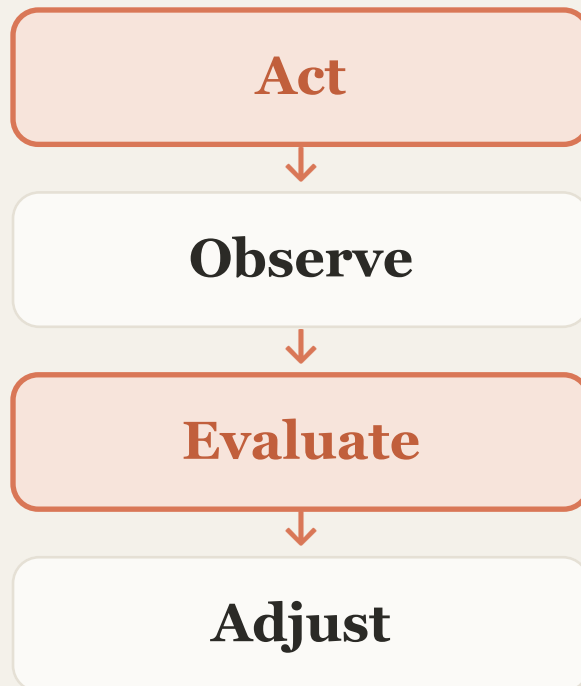
20 Workflow Optimization

The system redesigns itself.

It measures its own latency, cost, and quality — then reshapes its workflow. Too slow? Parallelize. Too costly? Downsize the model. Quality low? Add critic.



The pattern behind all 20



The output is never final on the first attempt. The loop is what turns a starting point into something production-worthy.



Better prompts get a better response.

Better loops get a system that improves itself — every day, without anyone touching it.

Agents are workers.

Loops are what make workers improve.



Found this useful?

- **Save it** for your next agent build
- **React** if you build loops, not just prompts
- **Share** with an AI engineer who needs it

For more AI agent insights

Subscribe to the newsletter:

rakeshgohel.substack.com/subscribe

Grab the book — The Human-Agent Orchestrator:

amzn.to/4gElzmv

